PART IV — FROM RAG TO AGENTIC RAG

Chapter 22C

Two-Track Parallel Compression in the LangGraph Rewrite

“Two tracks that never trust each other's chunks can still trust each other's order.”

Chapter 22C — Two-Track Parallel Compression in the LangGraph Rewrite

Everything in this chapter is the same NAC/DC/LBC machinery Chapter 22B already built, ported node by node the way Chapter 19B ported the agent loop itself — with one real architectural difference the two tracks can't share, and one precedence rule at the join point that decides what happens when they disagree.

22C.1 Why split into two tracks

`documents` chunks are noisy in a specific way: overlapping boundaries, redundant restatement across source files, and no guarantee any given chunk was ever seen or vetted before this exact query retrieved it. `learned_qa` chunks are noisy in a different, much smaller way: they are the project's own distilled, previously-validated question-answer pairs — Chapter 15's self-learning loop already ran a quality gate before anything reached this collection at all. Treating both tracks with identical compression intensity either under-cleans the noisy track or over-processes the already-clean one.

There is a second, independent reason to keep the tracks apart that has nothing to do with compression intensity: provenance. A document chunk and a learned-QA chunk answer a fundamentally different question about themselves — one says "here is what a source file states," the other says "here is what this system previously concluded and validated." Compressing them through one shared pipeline that has no notion of which track a chunk came from would make it structurally impossible to apply Chapter 22C.10's precedence rule at the end, because by the time two tracks' chunks are compressed together, the information about which track each one belonged to is already gone.

Chapter 11.5 made this same argument at the retrieval boundary — `retrieve_separate` returns two lists, never one merged and re-ranked list, for exactly this reason. Everything downstream of retrieval, compression included, inherits that same discipline: two tracks stay two tracks until an explicit, visible join point decides how to combine them, never through an implicit merge buried inside a shared processing step.

22C.2 The state-schema split

`GraphState` (Chapter 19B.4) declares the two tracks as parallel field families from the moment chunks are retrieved through the moment they're compressed, never merging early.

```
retrieved_document_chunks:    Annotated[list[dict], operator.add]
retrieved_learned_qa_chunks:  Annotated[list[dict], operator.add]
validated_document_chunks:    list[dict]
validated_learned_qa_chunks:  list[dict]
dedup_merged_document_chunks:   list[dict]
dedup_merged_learned_qa_chunks: list[dict]
nac_output_document_chunks:   list[dict]        # documents only — Section
22C.4
dc_output_document_chunks:   list[dict]
compressed_document_chunks:  list[dict]
dc_output_learned_qa_chunks:  list[dict]
compressed_learned_qa_chunks: list[dict]
compressed_docs: list[dict]                 # the join — Section 22C.9
```

Every field name carries its track in its own spelling — `_document_chunks` or `_learned_qa_chunks` — so a node reading the wrong track's field is a visible typo in a code review, not a silent cross-contamination bug waiting to be found in a debug log.

22C.3 The document track — the full pipeline

Documents run all three stages Chapter 22B built: `NAC_documents → DC_documents → LBC_documents`, in the graph exactly as `compress_context_pipeline` ran them in sequence. Nothing about the logic changes in the port — only its home, from three function calls inside one Python function to three registered graph nodes connected by edges. Running the full three-stage pipeline on documents specifically, and not on learned QA, is a direct consequence of where each collection's noise actually comes from. A document chunk's redundancy is structural — it comes from how the source was split and how many source files happen to restate the same fact — which is exactly the class of noise NAC and DC exist to remove. Nothing about a distilled Q&A pair carries that same structural redundancy, because nothing split it from a longer document in the first place.

22C.4 The learned_qa track — DC then LBC, NAC skipped

Learned-QA chunks skip NAC entirely, for a reason grounded directly in what NAC actually needs: neighbor merging requires a `chunk_seq` position within a source document, and a distilled Q&A pair — synthesized whole by `self_learner.py`, never split from a longer document — has no sequence to merge by. `execute_dc_learned_qa` reads straight from `dedup_merged_learned_qa_chunks`, the same field position `execute_dc_documents` reaches only after `nac_output_document_chunks` has already run.

```
# documents track
chunks = state.get("nac_output_document_chunks") or []
# learned_qa track — one stage earlier, no NAC output to read from
chunks = state.get("dedup_merged_learned_qa_chunks") or []
```

This is the asymmetric depth Figure 19B.2 already visualized structurally — three stages against two — grounded here in exactly why the depths differ: not an arbitrary optimization, but NAC having literally nothing to operate on for a track whose chunks were never chunked in the first place.

22C.5 Extracting each stage into its own node file

`nac.py`, `dc.py`, and `lbc.py` each hold both tracks' logic side by side — `dc.py` contains `execute_dc_documents` and `execute_dc_learned_qa` as two functions in one file, sharing a private `_run_dc` helper, rather than two separate files that would duplicate the scanning logic itself. `dedup_merge.py` and `combine_tracks.py` round out the set — one file per compression concept, not one file per track, which keeps the actual DC algorithm defined exactly once even though it runs on two independent inputs.

File	Document-track exports	Learned-QA-track exports
`nac.py`	`execute_nac_documents`, `validate_nac_documents`	— (skipped, Section 22C.4)
`dc.py`	`execute_dc_documents`, `validate_dc_documents`	`execute_dc_learned_qa`, `validate_dc_learned_qa`
`lbc.py`	`execute_lbc_documents`, `validate_lbc_documents`	`execute_lbc_learned_qa`, `validate_lbc_learned_qa`
`dedup_merge.py`	`validate_dedup_merge_documents`	`validate_dedup_merge_learned_qa`
`combine_tracks.py`	the single fan-in barrier for both tracks	the single fan-in barrier for both tracks

One file per stage rather than one file per track is a real design choice with a real cost avoided: a bug found in the DC scanning logic gets fixed once, in `_run_dc`, and both tracks inherit the fix on their next run. Splitting into `dc_documents.py` and `dc_learned_qa.py` would have made that same fix a two-file, easy-to-desynchronize change — the exact class of drift Chapter 19B.12's "maintain both pipelines side-by-side" discipline exists to prevent at the package level, recreated here at the file level if the split had gone the other way.

22C.6 The execute_X / validate_X pattern

> **Definition — Action/judgment node split**
> Separating a compression stage's mechanical work (execute_X) from its verdict-checking (validate_X) into two distinct graph nodes, connected by a conditional edge, rather than one node that always does both.

`execute_dc_documents` runs the scan and stashes its proposed redundancy groups into state (`dc_groups_per_window_documents`) without judging them at all. `validate_dc_documents` is a separate node entirely, reading that stashed state and running the actual judge calls. Splitting the two is what makes Section 22C.7's routing possible — a graph can only conditionally skip a stage if that stage is its own node with its own edge, and Chapter 22B's validators were never optional inside one Python function the way they are optional here.

22C.7 Per-stage routing functions

Three routing functions, one per compression-stage boundary, all sharing the identical shape: check the `ENABLE_COMPRESSION_VALIDATION` switch and route to the validator or skip straight past it.

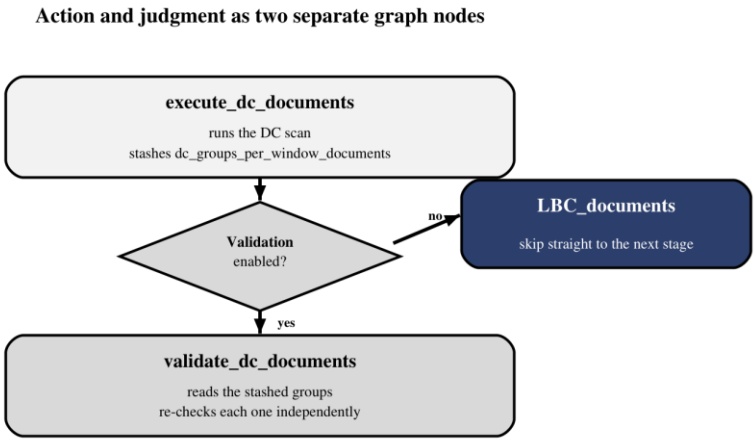

Figure 22C.1 — Validation is a real graph node, conditionally reachable, never a step a disabled switch quietly no-ops inside.

```
def route_dc_documents_to_validator(state: GraphState) -> str:
    return                    "validate_DC_documents"                    if
get_switches(state)["ENABLE_COMPRESSION_VALIDATION"] else "LBC_documents"

def route_dc_learned_qa_to_validator(state: GraphState) -> str:
    return                  "validate_DC_learned_qa"                   if
get_switches(state)["ENABLE_COMPRESSION_VALIDATION"] else "LBC_learned_qa"
```

Figure 22C.1's diamond is the same conditional-edge primitive Chapter 19.7 introduced, applied here per compression stage per track — six routing decisions total across both tracks' full pipelines, each one independently able to skip its validator without touching any other stage's wiring.

22C.8 Fan-out from query_variants to retrieve

Both tracks begin from the identical fan-out Chapter 19B.8 already built: `fan_out_retrievals` sends one `Send("retrieve", ...)` per query variant, and each parallel `retrieve` call queries *both* collections with one embedding via `retrieve_separate` (Chapter 11.5), writing into both tracks' reducer-typed fields simultaneously. The two tracks don't fan out separately — one fan-out populates both, and only diverge once `post_retrieval_filter` and the per-track validators run.

This shared fan-out point is worth noticing precisely because it is the *only* place the two tracks' execution is still coupled. From `post_retrieval_filter` onward, a document-track node and a learned-QA-track node share no data dependency at all — `execute_dc_documents` and `execute_dc_learned_qa` could run on entirely different machines and neither would need to know the other exists. LangGraph's scheduler exploits exactly this: with no edge connecting the two tracks' internal nodes, nothing forces them into the same superstep, and the framework is free to run whichever track's next node is ready first.

22C.9 Fan-in at combine_tracks

Both tracks' compression pipelines end at the same `defer=True` barrier Chapter 19B.10 built — `combine_tracks` waits for both `validate_LBC_documents` and `validate_LBC_learned_qa` to finish, regardless of which track's shorter path gets there first, then assembles one combined list.

```
learned_qa = state.get("compressed_learned_qa_chunks") or []
documents  = state.get("compressed_document_chunks") or []
combined = [*learned_qa, *documents]
return {"compressed_docs": combined}
```

22C.10 The conflict-resolution header

`compressed_docs` isn't just concatenated — the text handed to the model is wrapped with an explicit precedence rule and section labels, `format_precedence_context_for_llm` (Chapter 22B.7) applied at exactly this join point.

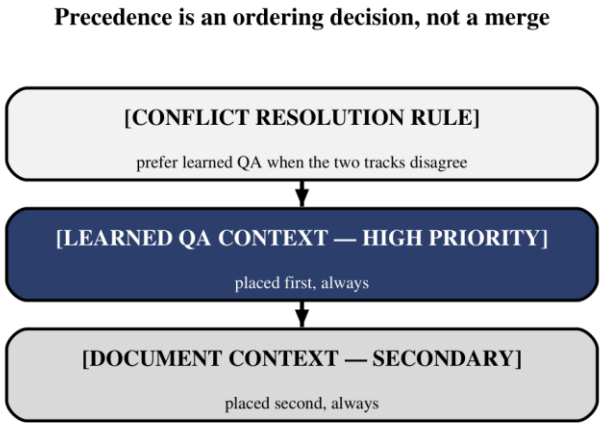

Figure 22C.2 — Learned QA is labeled and placed first, every time, not merely listed first by coincidence of concatenation order.

The ordering in Figure 22C.2 is the entire mechanism — no code branch decides per-query which track "wins" a conflict. The instruction is stated once, in the context itself, and the model resolves any actual conflict at generation time using the stated rule. Placing the instruction and the higher-priority section first is Chapter 14.1's recency-and-primacy discipline applied to context assembly instead of a system prompt: what the model reads first about how to treat a conflict is what it treats as authoritative.

Why learned QA outranks documents, specifically, is worth stating rather than leaving implicit: a learned-QA entry is not raw source material — it is a record of a question this system has already answered and had validated, by the same quality machinery Chapter 20 built. Preferring it over a freshly-retrieved document chunk when the two genuinely disagree is a bet that a previously-checked answer is more trustworthy than an unvalidated passage of source text, not a bet that learned QA is simply newer or more convenient to read. That reasoning is exactly what "HIGH PRIORITY" is standing in for in three words a prompt has room for, and it is worth remembering the full version the next time this precedence rule needs revisiting.

22C.11 The _THIN separator and per-track telemetry

Every node in both tracks logs through the same `_THIN` rule Chapter 22.2 established, and every log line names its track explicitly in its own tag — `[COMPRESS] running DC_documents on N chunk(s)` beside `[COMPRESS] running DC_learned_qa on N chunk(s)` — so two genuinely parallel streams of execution remain readable as two streams in one interleaved log file, rather than one confusing stream where a reader has to guess which track produced which line.

This is Chapter 22.6's per-iteration tool-call logging lesson generalized to true parallelism: a trace built for a sequential loop only ever had one thing happening at a time to label. A trace built for a graph with genuine concurrent branches has to label *which branch*, every time, or the trace stops being a reconstruction of what happened and becomes a shuffled deck of lines from two different stories.

The practical test is whether a reader can `grep` one track's story out of a run that interleaved both. `grep learned_qa run.debug.log` should return a coherent, ordered account of that track alone — every stage it passed through, every verdict its validators reached — with the document track's lines simply absent, not interspersed in a way that breaks the narrative. That property does not happen by accident; it happens because every node in both tracks was written, from the first line, to name its own track in every message it emits, the same discipline `caller_tag` (Chapter 13B.7) applies to LLM calls, extended here to cover an entire parallel branch instead of a single call site.

None of this — the field-name discipline, the shared-file-per-stage layout, the explicit routing, the stated precedence — was strictly required for the pipeline to produce a correct-looking answer on a happy path. A single merged track with no precedence rule would often return something reasonable too, right up until the one query where the two tracks genuinely disagreed and nothing in the system had ever decided, in advance, which one should be believed. Every seam this chapter added is a seam drawn before that moment arrives, not after a bad answer already shipped and someone had to reconstruct, after the fact, which track the wrong claim had come from.

Two tracks, one shared compression algorithm, one explicit precedence rule at the point they finally meet. Nothing about splitting documents from learned QA required inventing new compression logic — it required deciding, deliberately, which parts of Chapter 22B's pipeline the two tracks could share (the DC and LBC algorithms themselves) and which parts they couldn't (NAC's need for a sequence to merge by, and ultimately, which track's evidence wins when they disagree). Part V turns from the mechanics of one query to the resource every mechanism in this book has been spending all along: the token budget a context window has room for.